

Laporan Praktikum Kontrol Cerdas

Nama : Gunawan Wibisono
NIM : 224308083
Kelas : TKA – 7D
Akun *Github* (Tautan) : <https://github.com/gunawanwibisono83>
Student Lab Assistant : -

1. Judul Percobaan

Canny Edge Detection & Lane Detection with Instance Segmentation

2. Tujuan Percobaan

Tujuan dilakukannya percobaan ini yaitu :

- a. Mahasiswa dapat memahami konsep *Canny Edge Detection* sebagai metode dasar deteksi tepi.
- b. Mahasiswa dapat mengimplementasikan *Instance Segmentation* untuk deteksi jalur rel kereta (*Lane Detection*).
- c. Mahasiswa dapat menggabungkan metode *Canny Edge Detection* dengan *Instance Segmentation* untuk meningkatkan deteksi jalur.

3. Landasan Teori

Deep learning adalah salah satu bidang *machine learning* yang memanfaatkan banyak *layer* pengolahan informasi nonlinier untuk melakukan ekstraksi fitur, pengenalan pola, dan klasifikasi. Menurut Goodfellow, dkk. (2016), *deep learning* adalah sebuah pendekatan dalam penyelesaian masalah pada sistem pembelajaran komputer yang menggunakan konsep hierarki. Konsep hierarki membuat komputer mampu mempelajari konsep yang kompleks dengan menggabungkan dari konsep-konsep yang lebih sederhana (Rosita Dewi, 2018). Pada dasarnya *deep learning* adalah implementasi konsep dasar dari *machine learning* yang menerapkan algoritma ANN dengan lapisan yang lebih banyak (Alam dkk., 2019). Dalam bidang *Computer Vision*, *Deep Learning*

banyak digunakan untuk berbagai aplikasi seperti klasifikasi citra, deteksi objek, segmentasi citra, dan pengenalan wajah. Salah satu model *Deep Learning* yang populer adalah YOLO (*You Only Look Once*) yang digunakan untuk mendeteksi objek secara real-time dengan akurasi tinggi.

Deteksi tepi (*edge detection*) adalah teknik penting dalam pemrosesan citra digital yang digunakan untuk mengidentifikasi batas objek dalam gambar. Salah satu algoritma deteksi tepi yang banyak digunakan adalah *Canny Edge Detection*, yang diperkenalkan oleh John F. Canny pada tahun 1986. Algoritma ini dirancang untuk memenuhi tiga kriteria utama yaitu *good detection*, *good localization*, dan memastikan hanya ada satu respons untuk setiap tepi.

Proses deteksi tepi *Canny* terdiri dari beberapa tahapan. Pertama, gambar dihaluskan menggunakan filter *Gaussian* untuk mengurangi *noise*. Selanjutnya, gradien intensitas gambar dihitung untuk menyoroti area dengan perubahan intensitas yang tinggi. Setelah itu, dilakukan *non-maximum suppression* untuk menipiskan tepi yang terdeteksi sehingga hanya puncak gradien yang dipertahankan. Terakhir, algoritma menerapkan *hysteresis thresholding* dengan dua ambang batas untuk menentukan tepi akhir, memastikan bahwa tepi yang lemah namun terhubung dengan tepi yang kuat tetap dipertahankan. Keunggulan utama dari Algoritma *Canny* adalah kemampuannya dalam mendeteksi tepi dengan akurasi tinggi dan ketahanan terhadap *noise*. Hal ini menjadikannya pilihan populer dalam berbagai aplikasi, seperti pengenalan objek, analisis citra medis, dan sistem *Computer Vision* (Nugroho dkk., 2025).

Instance segmentation merupakan teknik segmentasi citra hingga ke level piksel. *Instance segmentation* digunakan agar dapat mendeteksi beberapa objek yang berada pada kelas sama maupun beda sebagai sebuah individual objek. Teknik ini menggabungkan elemen-elemen dari teknik klasik *computer vision* yaitu *object detection* dengan cara mengklasifikasikan tiap objek pada citra, dan *localize* tiap objek menggunakan *bounding box* dan *semantic segmentation*. Setiap hasil deteksi memberikan beberapa informasi seperti kelas dari tiap objek yang ditemukan, dan lokasi ditemukannya objek tersebut. Dalam konteks deteksi jalur rel, *Instance Segmentation* berfungsi untuk mengenali dan memisahkan antara rel utama (*Main Rail*) dan rel cabang (*Branch Rail*) (Bergmann, 2024).

4. Analisis

Dalam praktikum ini, praktika melakukan implementasi *Canny Edge Detection* dan *Instance Segmentation* menggunakan YOLOv8, serta melakukan penggabungan keduanya untuk mendeteksi jalur rel kereta pada suatu gambar. Tujuan dari percobaan ini adalah untuk membandingkan kemampuan kedua metode dalam mendeteksi struktur rel serta menganalisis efek kombinasi keduanya terhadap hasil deteksi dan akurasi visual.

Metode *Canny Edge Detection* bekerja dengan mendeteksi perubahan intensitas piksel secara bertahap sehingga menghasilkan garis tepi (*edge*) yang merepresentasikan batas objek. Dengan parameter *threshold* bawah sebesar 50 dan *threshold* atas sebesar 150, serta penggunaan filter *Gaussian* kernel 5×5 untuk mengurangi *noise*, metode ini mampu menampilkan garis tepi rel secara cukup jelas pada kondisi pencahayaan dan kontras yang baik. Namun, kekurangannya adalah metode ini tidak mampu membedakan antara objek rel dan objek lain di sekitar seperti bebatuan, bayangan, atau elemen lingkungan lain. Akibatnya, hasil deteksi masih menampilkan banyak garis yang tidak relevan atau terputus pada bagian tertentu dari rel.

Sebaliknya, metode *Instance Segmentation* berbasis YOLOv8 menunjukkan hasil deteksi yang lebih presisi karena model ini telah dilatih untuk mengenali dua kelas objek secara spesifik, yaitu *Main Rail* dan *Branch Rail*. Hasil segmentasi tidak hanya menampilkan tepi objek, tetapi juga area (*mask*) rel secara menyeluruh dengan warna berbeda untuk masing-masing kelas. Dari pengamatan visual, segmentasi YOLOv8 mampu mendeteksi rel dengan tingkat keakuratan yang lebih baik, bahkan pada kondisi gambar yang kompleks. Dibandingkan *Canny*, hasil dari YOLOv8 terlihat lebih halus dan lebih fokus pada area yang memang merupakan jalur rel.

Ketika kedua metode tersebut dikombinasikan, yakni dengan melakukan *overlay* hasil segmentasi YOLOv8 dan tepi hasil *Canny*, sistem mampu menampilkan deteksi yang lebih sesuai secara visual. *Canny* menonjolkan struktur tepi rel dengan jelas, sementara YOLOv8 memberikan area segmentasi yang nyata. Kombinasi ini menghasilkan citra dengan representasi visual yang lebih informatif dimana area rel terlihat menyeluruh dan garis batasnya lebih

tegas. Berdasarkan hasil observasi, kombinasi kedua metode ini juga membantu mengurangi *false detection* karena area yang bukan rel dapat diabaikan berkat segmentasi YOLOv8.

Selanjutnya, terhadap pengaruh perubahan parameter *Canny*, dapat disimpulkan bahwa nilai *threshold* sangat memengaruhi sensitivitas deteksi tepi. Ketika *threshold* bawah dan atas diperbesar (misalnya 75 dan 200), hanya tepi-tepi yang memiliki kontras tinggi yang terdeteksi sehingga garis rel terlihat lebih sedikit namun lebih bersih dari *noise*. Sebaliknya, jika nilai *threshold* lebih kecil (misalnya 30 dan 100), lebih banyak garis terdeteksi termasuk *noise*, sehingga hasil menjadi lebih padat tetapi kurang spesifik. Oleh karena itu, pemilihan parameter *threshold* harus disesuaikan dengan tingkat pencahayaan dan kualitas gambar agar hasil deteksi optimal.

Secara keseluruhan, hasil percobaan menunjukkan bahwa *Instance Segmentation* menggunakan YOLOv8 memiliki akurasi yang jauh lebih baik dibandingkan *Canny Edge Detection* dalam mengenali struktur rel. Namun, kombinasi kedua metode tersebut memberikan hasil terbaik secara visual, karena mampu memanfaatkan keunggulan segmentasi area dari YOLOv8 dan kejelasan garis tepi dari *Canny*.

5. Diskusi

Metode *Canny Edge Detection* lebih baik digunakan pada kondisi di mana diperlukan deteksi cepat terhadap struktur garis atau batas objek, seperti pada tahap prapemrosesan citra untuk menandai bentuk rel tanpa memerlukan model pembelajaran yang kompleks. Sementara itu, *Instance Segmentation* lebih unggul untuk mengenali dan mengklasifikasi area objek secara spesifik, terutama bila sistem membutuhkan pemahaman konteks seperti posisi rel terhadap lingkungan sekitar.

Akurasi deteksi jalur dengan YOLOv8-seg dapat ditingkatkan melalui *tuning* parameter seperti ukuran *input* gambar (*imgsz*), jumlah *epochs*, serta proses *fine-tuning* model menggunakan dataset rel kereta yang lebih spesifik agar bobot jaringan dapat menyesuaikan karakteristik visual rel.

Dalam konteks sistem navigasi kereta otomatis, kombinasi kedua metode ini dapat diterapkan untuk meningkatkan keandalan sistem. YOLOv8 dapat mengenali jalur utama dan objek penghalang, sedangkan *Canny* dapat membantu mempertegas batas rel untuk memastikan posisi dan arah laju kereta terdeteksi dengan presisi tinggi.

6. Assignment

Setelah melakukan praktikum implementasi dasar *Canny Edge Detection* dan *Instance Segmentation*, praktikum dilanjutkan dengan *assignment* perubahan parameter dan modifikasi model deteksi untuk mendeteksi rel saja. Percobaan pertama difokuskan pada perubahan parameter pada metode *Canny Edge Detection* dengan tiga kombinasi nilai (threshold) dan ukuran kernel *Gaussian* yang berbeda. Tujuannya adalah untuk melihat bagaimana sensitivitas tepi dan tingkat kehalusan deteksi dipengaruhi oleh perubahan parameter tersebut.

Pada pertama, digunakan nilai *low threshold* = 25, *high threshold* = 75, dan ukuran kernel 3×3 . Hasil yang diperoleh menunjukkan bahwa garis tepi yang terdeteksi sangat banyak dan cenderung tidak stabil, termasuk banyak *noise* di area non-rel. Hal ini disebabkan oleh nilai ambang yang terlalu rendah, sehingga metode *Canny* menjadi terlalu sensitif terhadap perubahan kecil pada intensitas cahaya. Walaupun struktur rel masih terlihat, batasnya kurang jelas dan hasil deteksi tampak berantakan. Selanjutnya, parameter diubah menjadi *low threshold* = 75, *high threshold* = 200, dan kernel 7×7 . Dengan nilai ambang yang lebih tinggi dan kernel yang lebih besar, hasil deteksi menjadi lebih bersih dan fokus pada garis-garis utama dari rel. Sebagian besar *noise* dapat dihilangkan, namun beberapa bagian rel yang memiliki kontras rendah tidak terdeteksi secara utuh. Kondisi ini menunjukkan bahwa peningkatan nilai ambang dapat mengurangi sensitivitas sistem terhadap detail kecil, tetapi mampu mempertegas struktur utama dari objek. Pada parameter ketiga, digunakan *low threshold* = 100, *high threshold* = 250 dengan ukuran kernel 7×7 . Hasil yang diperoleh menunjukkan bahwa deteksi tepi menjadi lebih minimalis dimana hanya bagian rel dengan kontras tinggi yang terdeteksi. Meskipun garis rel tampak jelas,

sebagian area rel yang redup tidak terdeteksi sama sekali. Hal ini mengindikasikan bahwa semakin tinggi nilai ambang yang digunakan, semakin selektif sistem dalam mendeteksi tepi namun dengan konsekuensi kehilangan detail penting. Dari ketiga pengujian tersebut, kombinasi nilai *threshold* = 50 dan 150 dengan kernel 5×5 dianggap paling seimbang antara tingkat sensitivitas dan kehalusan tepi.

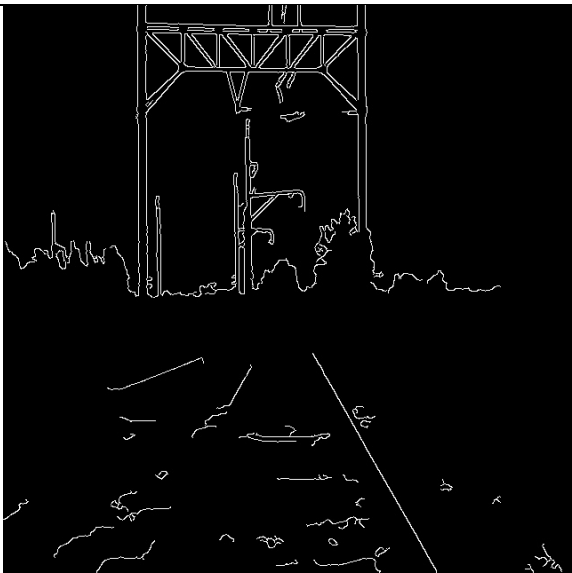
Pada *assignment* berikutnya, dilakukan modifikasi kode *Instance Segmentation* menggunakan YOLOv8-seg, dengan model hasil *training* dari dataset yang telah dilabeli 2 kelas menjadi satu kelas yaitu *Rail*. Hasil deteksi menunjukkan bahwa model mampu mengenali area rel dengan cukup akurat. Area rel divisualisasikan dalam bentuk *mask* berwarna yang mengikuti bentuk jalur rel. Metode ini terbukti lebih unggul dibandingkan *Canny Edge Detection* dalam hal presisi area dan konsistensi hasil, karena model bekerja berdasarkan pembelajaran fitur visual yang kompleks, bukan hanya berdasarkan perubahan intensitas piksel.

Terakhir, kedua metode tersebut digabungkan menjadi sistem *Combined Detection*, yaitu menggabungkan hasil *Canny Edge Detection* dengan *Instance Segmentation* (YOLOv8-seg). Pada kombinasi ini, parameter *Canny* menggunakan *low threshold* = 50, *high threshold* = 150, dan kernel 5×5. Hasilnya menunjukkan bahwa kombinasi tersebut mampu menghasilkan deteksi rel yang lebih informatif dimana area rel teridentifikasi dengan jelas melalui segmentasi, sementara garis batas rel ditegaskan oleh hasil *Canny*.

7. Data dan Output Hasil Pengamatan

No.	Keterangan	Output
1.	<i>Output Canny Edge Detection</i> dengan <i>low threshold</i> = 50, <i>high threshold</i> = 150 dan ukuran kernel 5x5.	

2.	Output dari implementasi <i>Lane Detection</i> dengan <i>Instance Segmentation</i> (YOLOv8-seg) yang mengacu berdasarkan hasil <i>training</i> dari dataset yang telah dilabeli 2 <i>bounding box</i> yaitu <i>main rail</i> dan <i>branch rail</i>.	 The image shows a real-time detection window titled "YOLOv8 Rail Detection - Real Time". It displays a perspective view of railway tracks. Two sets of bounding boxes are overlaid: blue boxes for "Branch Rail" and cyan boxes for "Main Rail". The confidence scores are: Branch Rail 0.86, Branch Rail 0.95, and Main Rail 0.92.
3.	Output dari gabungan <i>Lane Detection</i> dengan <i>low threshold</i> = 50, <i>high threshold</i> = 150 dan ukuran kernel 5x5 dan <i>Instance Segmentation</i> (YOLOv8-seg) dengan mengacu berdasarkan hasil <i>training</i> dari dataset yang telah dilabeli 2 <i>bounding box</i> yaitu <i>main rail</i> dan <i>branch rail</i> tanpa menampilkan <i>bounding box</i>.	 The image shows the same railway track scene as in row 2, but without the bounding boxes. The rail segments are highlighted with colors: the left branch rail is purple and the main rail is cyan, representing the output of the combined detection process.
4.	Output <i>Canny Edge Detection</i> dengan <i>low threshold</i> = 25, <i>high threshold</i> = 75 dan ukuran kernel 3x3.	 The image shows the Canny edge detection result of the railway track scene. The edges of the tracks, the overhead power lines, and the surrounding landscape are highlighted in white against a black background.

5.	<i>Output Canny Edge Detection dengan low threshold = 75, high threshold = 200 dan ukuran kernel 7x7.</i>	
6.	<i>Output Canny Edge Detection dengan low threshold = 100, high threshold = 250 dan ukuran kernel 7x7.</i>	
8.	<i>Output dari implementasi Lane Detection dengan Instance Segmentation (YOLOv8-seg) dengan mengacu berdasarkan hasil training dari dataset yang telah dilabeli 1 bounding box yaitu rail.</i>	

6.	Output dari gabungan <i>Lane Detection</i> dengan <i>low threshold</i> = 50, <i>high threshold</i> = 150 dan ukuran kernel 5x5 dan <i>Instance Segmentation</i> (YOLOv8-seg) dengan mengacu berdasarkan hasil <i>training</i> dari dataset yang telah dilabeli 1 <i>bounding box</i> yaitu rail tanpa menampilkan <i>bounding box</i>.	
----	---	--

8. Kesimpulan

Berdasarkan percobaan yang telah dilakukan, maka dapat diambil kesimpulan sebagai berikut:

- Mahasiswa dapat memahami konsep *Canny Edge Detection* sebagai metode dasar deteksi tepi, mengimplementasikan *Instance Segmentation* untuk deteksi jalur rel kereta (*Lane Detection*), serta menggabungkan metode *Canny Edge Detection* dengan *Instance Segmentation* untuk meningkatkan deteksi jalur.
- Pada praktikum deteksi tepi, *Canny Edge Detection* lebih cocok untuk menyoroti garis batas objek namun sangat bergantung pada kualitas pencahayaan dan parameter *threshold*.
- Sedangkan pada *Instance Segmentation* (*Lane Detection*) YOLOv8 memberikan hasil deteksi yang lebih akurat dan stabil karena memanfaatkan proses pembelajaran yang lebih mendalam. Hasil segmentasi tidak hanya menampilkan tepi objek, tetapi juga area (*mask*) rel secara menyeluruh.
- Kombinasi keduanya memberikan hasil paling optimal secara visual, di mana segmentasi memperlihatkan area rel secara utuh dan *Canny* mempertegas tepinya.

9. Saran

Sebagai saran, pada percobaan selanjutnya disarankan untuk performa dan akurasi program dapat dioptimalkan dengan memanfaatkan GPU, sehingga prediksi YOLO dapat berjalan lebih cepat dan responsif. Selain itu, penerapan preprocessing tambahan serta pemilihan parameter yang sesuai pada *Canny Edge Detection* dapat membantu mengurangi *noise* yang tidak diinginkan. Untuk meningkatkan keterbacaan visual, transparansi *overlay* warna dapat disesuaikan agar objek yang terdeteksi lebih jelas tanpa menutupi detail penting dalam gambar.

10. Daftar Pustaka

- Alam, I. F., Sarita, M. I., & Sajiah, A. M. (2019). IMPLEMENTASI DEEP LEARNING DENGAN METODE CONVOLUTIONAL NEURAL NETWORK UNTUK IDENTIFIKASI OBJEK SECARA REAL TIME BERBASIS ANDROID. *semanTIK*, 5(2), 237–244. <https://doi.org/10.5281/zenodo.3459374>
- Bergmann, D. (2024). *Apa itu segmentasi instance?* Artificial Intellegence. <https://www.ibm.com/id-id/think/topics/instance-segmentation>
- Nugroho, C. W., Nurtanio, I., & Jalil, A. (2025). Penentuan Kualitas Kopra Berbasis Citra Kontur Menggunakan Metode Canny Edge Detection: Determination of Copra Quality Based on Contour Image Using the Canny Edge Detection Method. *MALCOM: Indonesian Journal of Machine Learning and Computer Science*, 5(1), 436–450. <https://doi.org/10.57152/malcom.v5i1.1823>
- Rosita Dewi, S. (2018). *DEEP LEARNING OBJECT DETECTION PADA VIDEO MENGGUNAKAN TENSORFLOW DAN CONVOLUTIONAL NEURAL NETWORK* [Universitas Islam Indonesia]. https://dspace.uii.ac.id/bitstream/handle/123456789/7762/14611242_Syarifah%20Rosita%20Dewi_Statistika.pdf?sequence=1&isAllowed=y